

Control Properties

There are many widget properties that you can change, using Glade. The most important property, from a code perspective, is the Name property, accessible on the General tab of the Properties pane. We have renamed all the widgets that we are going to access from code, providing names that follow our widget-naming convention. For example, the text entry names are `login_ent` and `pw_ent`; the buttons are named `ok_btn` and `cancel_btn`. You don't necessarily need to follow this convention, but when doing team development, it's important that you follow the scheme chosen for the project, or used by the rest of the development team.

Under the Common tab, you can change the width and height requests of widgets. You can add tool-tips, and control whether the widget is visible (shown when its container is shown) or not. Further, you can change the Sensitivity setting—whether the widget responds to user input or not.

The 'data' property of a widget that is displayed to the user varies in name from widget to widget, and has a convention that follows the purpose of the control, and the type of data that is displayed. For example, a text entry widget has a Text property that represents the (editable) text shown to the user. A label control has the Label property, in which you enter the (non-editable) text you want the label to display to the user.

Many widgets, by default, fill all the available space in the parent container when they are added. If you find this annoying, you can change this behaviour: select the widget, and in the Properties pane, go to the Packing tab. Modify the Expand and Fill settings that are available for most controls. You can change these properties from code as well.

Keyboard accelerators are useful for those using large windows, such as data entry forms; they enable the user to quickly activate controls like text entries or buttons, without having to pick up the mouse and click that control. Generally, these accelerators are alpha-numeric keys combined with the Alt key—for example, `Alt+S` is a common accelerator for a search button. Generally, any widget with a label can have a keyboard accelerator. To add one, click the widget in the object tree; in the Properties pane's General tab, go to the Label property. Add an underscore just before the letter you want to use as the accelerator (for example, on a search button, the label would become `_Search`). Toggle the Use underline property from No to Yes. When the window is running, pressing the respective accelerator will activate the widget for which you have set the accelerator key. Obviously, avoid assigning the same accelerator key to two controls on the same window!

3. Click on *GtkWidget* to expand it.
4. Click on delete-event, and click under *Handler* to reveal the pull-down menu.
5. Choose `on_window1_delete_event` as the name for the handler.
6. Leave `user_data` blank; you do not need it.
7. Leave the 'after' unchecked. You do not need to make this handler run after the default handler.
8. Save the Glade file.

Repeat this procedure to set up signal handlers for the OK button-click (`on_ok_btn_clicked`) and the Cancel button-click (`on_cancel_btn_clicked`). For the *destroy* event of the *oplevel* window, you must choose *GtkObject* instead of *GtkWidget*.

As we have just seen above, the hierarchy between the *GtkWidget* and *GtkObject* classes lets us handle two different signals for the same event (the user clicking the window's close button).

- **on_window1_delete_event:** This is triggered when the *window1 GtkWidget* is deleted. The return value from the signal handler we will write controls the propagation of the signal: *TRUE* means 'do not propagate the event to higher levels' and *FALSE* means 'the event should be propagated'. If the window has unsaved data, here is where you would prompt the user with a *Yes/No/Cancel* type of message dialog, asking whether to save the data and exit (*Yes* button), exit without saving (*No* button), and abort the closing of the window (*Cancel* button).
- **on_window1_destroy:** This is triggered when the *window1 GtkObject* is destroyed. If this was the last open window in the application (which it is for us!), in this handler, we must compulsorily execute code to exit the *Gtk2->main()* main-loop; otherwise the GUI *oplevel* window will be destroyed, but the application itself will not exit—it will continue running. To stop the *Gtk2* main loop, we call the *Gtk2->main_quit()* function in this handler.

Writing the glue code

First, we need the *Gtk2* Perl module (and its dependencies) installed, in order to use the GUI. You can use this command to install them:

```
$ sudo yum install perl-Gtk2.x86_64
```



Note: The LFY CD this month carries the entire Perl script. Take a look at Listing 1 in this CD.

The first step in a Perl GTK program is to load the *Gtk2* module, and that is done with:

```
use Gtk2 qw( -init );
```